

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG TP.HCM

PHIẾU NHẬN XÉT & ĐỊNH HƯỚNG PHÁT TRIỂN ĐỀ TÀI

Dự án μEDP — Micro Event-Driven Platform cho vi điều khiển

(Đánh giá khả năng phát triển thành đồ án / khóa luận tốt nghiệp)

Giảng viên: Ths. Nguyễn Duy Xuân Bách

Hạng mục	Nội dung
Dự án được đánh giá	μEDP (Micro Event-Driven Platform), tiền thân CIEDPC — framework lập trình hướng sự kiện theo mô hình Active Object cho hệ nhúng. Repo: github.com/1811htsang/uEDP
Nhóm thực hiện	Nguyễn Hoàng Hải Minh (contributor từ v1.1.5)
Người nhận xét	ThS. Nguyễn Duy Xuân Bách
Ngày lập phiếu	14/08/2026
Kết luận nhanh	ĐỦ ĐIỀU KIỆN phát triển thành đề tài — khuyến nghị định hướng khóa luận tốt nghiệp (có sản phẩm demo + tích hợp AI). Chi tiết ở Mục 6.

Mục lục

Mục lục	2
1. Nhận xét tổng quan về dự án	3
2. Kết luận về mức độ sẵn sàng làm đề tài	3
3. Điểm mạnh nổi bật	3
4. Điểm cần bổ sung / hoàn thiện	4
6. Định hướng khai thác thành đề tài (trọng tâm)	5
6.1. Hướng A — μEDP làm runtime cho Edge AI / TinyML ★ KHUYẾN NGHỊ	5
6.2. Hướng B — AI hỗ trợ sinh & tối ưu firmware (μE-LS + AI)	5
6.3. Hướng C — AI giám sát & dự đoán lỗi hệ thống (self-monitoring)	6
6.4. Bảng tổng hợp & gợi ý phân chia 2 thành viên	6
7. Khung báo cáo chi tiết (mẫu cho khóa luận)	7
8. Tiêu chí nghiệm thu (checklist)	8
9. Kết luận & bước tiếp theo	8

1. Nhận xét tổng quan về dự án

Sau khi khảo sát toàn bộ mã nguồn và hệ thống tài liệu của dự án, tôi ghi nhận đây **không phải một bài tập lớn thông thường** mà là một **framework hệ nhúng thực thụ, có chiều sâu kiến trúc**. μEDP đi theo mô hình **Active Object / event-driven** — cùng họ với QP/C của Miro Samek, cạnh tranh triết lý thiết kế với FreeRTOS.

Quy mô & mức độ hoàn thiện (số liệu khảo sát)

Thành phần	Quy mô	Ghi nhận
Lõi C (core + PAL + common + app + test)	≈ 6.400 dòng	Được viết chín chu, có chú thích, phân tầng rõ
Toolchain sinh code (Python)	≈ 700 dòng	Kconfig → Jinja2, tách 2 giai đoạn (PLTF)
Tài liệu thiết kế	20+ file	arch-design (VN+EN), so sánh QP/C & FreeRTOS, roadmap, user-manual
Lịch sử phát triển	v1.0.0 → v1.1.5+	Roadmap versioned, to-do tracking chi tiết theo từng phiên bản
Nền tảng hỗ trợ (theo thiết kế)	STM32, ESP32, Linux	PAL trừu tượng hóa phần cứng, hướng "Zero-Touch Porting"

Kiểm chứng trực tiếp trên mã nguồn (không chỉ đọc tài liệu): bộ lập lịch dùng **bitmap ưu tiên tra cứu O(1)**, cơ chế **run-to-completion**, **message pool tĩnh deterministic**, **hybrid state machine TSM+FSM**, **ISR bridge kiểu "delayed mapping"** (ngắt chỉ ghi thô, việc cấp phát tin nhắn đẩy ra ngoài ngữ cảnh ngắt), và **pipeline logging 3 tầng**. Đây là tư duy kiến trúc ở mức vượt khá xa mặt bằng sinh viên đại học.

Đánh giá năng lực

Nhóm thực sự HIỂU mô hình Active Object và các ràng buộc của hệ nhúng thời gian thực (xác định bộ nhớ, an toàn ngắt, độ trễ) — không phải sao chép rời rạc. Đây là nền tảng rất tốt để nâng lên đề tài học thuật.

2. Kết luận về mức độ sẵn sàng làm đề tài

Kết luận: Đủ điều kiện, và thực tế đã vượt ngưỡng “đề án môn học”. Vấn đề không còn là “có làm được đề tài không” mà là “định vị ở cấp độ nào và cắt phạm vi ra sao cho vừa sức bảo vệ”.

Cấp độ	Mức độ phù hợp	Điều kiện then chốt
Đề án môn học / NCKH cơ sở	Đã dư sức	Có thể dùng ngay bản hiện tại + bổ sung đo đạc cơ bản
Khóa luận tốt nghiệp	RẤT phù hợp (khuyến nghị)	Phải bổ sung: (1) sản phẩm demo chạy hoàn chỉnh, (2) tích hợp AI, (3) thực nghiệm đo đạc. Xem Mục 6.
Hướng công bố khoa học (báo/hội nghị)	Khả thi có điều kiện	Cần một đóng góp mới sắc bén + benchmark nghiêm túc (không chỉ “làm lại QP/C”)

Lưu ý quan trọng về phạm vi

Rủi ro lớn nhất của dự án này KHÔNG phải “thiếu” mà là “QUÁ TỒI”. Roadmap đang hướng tới cả một hệ điều hành (μE-OS). Với khóa luận, cần cắt xuống một lát cắt bảo vệ được trong 1 học kỳ, đừng ôm trọn μE-OS.

3. Điểm mạnh nổi bật

- **Codebase thật, có chiều sâu:** ≈6.400 dòng C, kiến trúc 3 tầng (App – Core – PAL) tách bạch, lỗi bất biến, dễ port.
- **Tư duy kiến trúc trưởng thành:** scheduler O(1) bằng bitmap, RTC, deterministic memory pool, ISR-safe delayed mapping, hybrid TSM/FSM.
- **Kỷ luật kỹ thuật hiếm thấy ở SV:** roadmap versioned, to-do tracking theo từng phiên bản, tài liệu thiết kế song ngữ, tham chiếu tài liệu gốc (TinyGALS, Protothreads, Active Objects, P language).
- **Có tư duy so sánh/định vị:** chủ động đối chiếu với chuẩn QP/C (v8.2.1) và FreeRTOS — hiểu mình đứng ở đâu.
- **Hạ tầng công cụ tốt:** sinh code Kconfig→Jinja2 (μE-LS), Docker hóa môi trường build (kèm ESP-IDF) — nền rất tốt cho model-driven development.
- **Tính tự chủ cao:** nhóm tự vận hành, tự lập kế hoạch — giảm đáng kể công sức hướng dẫn.

4. Điểm cần bổ sung / hoàn thiện

Các điểm dưới đây không làm giảm giá trị dự án — chúng chính là “phần việc” biến một framework tốt thành một khóa luận/bài báo bảo vệ được, và là nơi vai trò hướng dẫn phát huy giá trị.

#	Vấn đề	Yêu cầu bổ sung
1	Độ mới so với QP/C	QP/C đã là chuẩn vàng. Các khác biệt (hybrid TSM/FSM, pool lấy sizeof(void*) làm đơn vị, tách ngữ cảnh ISR) hiện ở mức BIẾN THỂ kỹ thuật. Cần chỉ rõ đâu là đóng góp mới thật sự — phần μE-LS (model-driven codegen) và tích hợp AI là hướng có novelty cao nhất.
2	Thiếu thực nghiệm đo đạc	Tài liệu so sánh ghi “Đạt/Vượt” nhưng CHƯA có số đo. Cần bảng thực nghiệm: latency, jitter, footprint RAM/Flash — đo trên phần cứng thật, so với FreeRTOS / bare-metal. Đây là điểm yếu cố hữu cần xử lý dứt điểm.
3	Repo public đang hụt so với tài liệu	README/tài liệu nêu 5 port (STM32 F103/H723, ESP32 WROOM/S3, Linux) nhưng nhánh main CHỈ có port Linux; lịch sử git bị nén thành 1 commit “dummy”. Cần push đủ các port MCU thật + giữ lịch sử commit thật để chứng minh khả năng “porting lên vi điều khiển” và minh bạch tiến trình.
4	Phân chia đóng góp 2 thành viên	Đề tài 2 người → phải chốt rõ ai sở hữu mất xích nào để chấm & bảo vệ độc lập (xem gợi ý Mục 6.4).
5	Xác minh mức độ tự chủ	Một phần tài liệu có dấu hiệu hỗ trợ AI. Không sao — nhưng nên kiểm tra hiểu bài qua vấn đáp nội tại scheduler / memory pool để bảo đảm nhóm làm chủ hoàn toàn.

6. Định hướng khai thác thành đề tài (trọng tâm)

Hai yêu cầu bắt buộc của khóa luận tốt nghiệp (định hướng của GV)

- ① **Sản phẩm có đầu ra thực tế**: một project chạy được, demo hoàn chỉnh trên phần cứng — không dừng ở framework/thư viện.
- ② **Tích hợp AI**: có thành phần trí tuệ nhân tạo (xử lý dữ liệu, tiên đoán, tối ưu, thuật toán AI) — bám xu hướng hiện nay.

Điểm mấu chốt: μEDP hiện là **hạ tầng (runtime)** — bản thân nó chưa phải “sản phẩm demo” và chưa có AI. Cách nâng lên khóa luận là **dùng μEDP làm lớp nền điều phối cho một ứng dụng biên (edge) có AI chạy được trên MCU**. Framework trở thành “động cơ”, còn AI + ứng dụng là “sản phẩm” thầy nhìn thấy và bấm chạy được. Dưới đây là 3 hướng, xếp theo mức khuyến nghị.

6.1. Hướng A — μEDP làm runtime cho Edge AI / TinyML ★ KHUYẾN NGHỊ

Vì sao khuyến nghị cho KLTN

Thỏa mãn ĐỒNG THỜI cả hai yêu cầu (sản phẩm demo chạy được + AI thật), rủi ro thấp nhất, và bám đúng xu hướng Edge AI / IoT Gateway.

Ý tưởng: Nhúng một mô hình học máy (TinyML) chạy **inference ngay trên vi điều khiển** — tổ chức pipeline “đọc cảm biến → tiền xử lý → suy luận AI → hành động/hiển thị”, trong đó **mỗi khối là một Active Object task điều phối bởi μEDP**. Demo trên STM32/ESP32 với cảm biến thật.

Sản phẩm demo cụ thể (chọn 1 để bảo vệ)

- **Predictive maintenance**: đọc rung động (cảm biến MEMS/accelerometer) → phân loại trạng thái máy bình thường / bất thường (anomaly detection). Rất “bán được”, dữ liệu dễ tạo.
- **Nhận dạng cử chỉ / từ khóa (gesture / keyword spotting)**: micro/IMU → CNN nhỏ phân loại real-time.
- **Node nông nghiệp/y tế thông minh**: cảm biến môi trường hoặc tín hiệu sinh học cơ bản → phân loại trạng thái on-device (khớp hướng Edge AI của nhóm).

Thành phần AI & thực nghiệm

- Mô hình MLP/CNN nhỏ, **lượng tử hóa int8**; pipeline train offline (Python/TensorFlow) → convert (TFLite Micro / CMSIS-NN) → deploy lên MCU.
- **Đóng góp học thuật (điểm bán thành báo)**: benchmark μEDP như một runtime cho AI-at-edge — đo latency/jitter/RAM khi inference chạy XEN KẾ các task thời gian thực, so với chạy trên FreeRTOS / bare-metal.

Đầu ra bảo vệ

Thiết bị thật đọc cảm biến → AI phán đoán → hiển thị/điều khiển real-time; kèm bảng số liệu accuracy + latency on-device và so sánh baseline.

6.2. Hướng B — AI hỗ trợ sinh & tối ưu firmware (μE-LS + AI)

Ý tưởng: Mở rộng μE-LS (YAML → sinh firmware). Thêm lớp AI theo 1 trong 2 nhánh: (a) **LLM code synthesis** — từ mô tả cấp cao/ngôn ngữ tự nhiên → sinh spec μE-LS → sinh code + test; hoặc (b) **AI auto-tuning** — dùng ML (Bayesian optimization / RL) gợi ý cấu hình tối ưu (kích thước pool, phân bổ task/priority) từ dữ liệu profiling.

- **Sản phẩm demo**: nhập mô tả → hệ thống sinh ra project μEDP build được và chạy trên board.
- **Mức độ**: rất trendy, novelty cao, hợp NCKH/báo. **Rủi ro**: dễ thành “AI wrapper” hời hợt nếu thiếu đánh giá định lượng — cần tiêu chí đo chất lượng code sinh ra.
- **Phù hợp**: thành viên mạnh về tooling/Python.

6.3. Hướng C — AI giám sát & dự đoán lỗi hệ thống (self-monitoring)

Ý tưởng: Khai thác telemetry runtime từ **itnlog** (task/signal/timing) → mô hình ML **phát hiện bất thường / dự đoán task treo, cạn tài nguyên trước khi xảy ra** → kích hoạt safe-response (kết hợp “Task Counter” đã có trong roadmap).

- **Sản phẩm demo:** hệ thống tự giám sát; chen lỗi giả lập → AI cảnh báo sớm / tự phục hồi.
- **Mức độ:** đóng góp = observability + predictive fault management cho AO runtime — hợp NCKH/báo.

6.4. Bảng tổng hợp & gợi ý phân chia 2 thành viên

Hướng	Sản phẩm demo	Thành phần AI	Định vị phù hợp
A ★	Node biên đọc cảm biến + inference real-time	TinyML on-device (CNN/MLP int8)	KLTN (tốt nhất) → mở rộng báo
B	Trình sinh firmware từ mô tả cấp cao	LLM code-gen / ML auto-tuning	NCKH / báo (novelty cao)
C	Hệ tự giám sát & phục hồi	Anomaly detection / time-series	NCKH / báo

Gợi ý chia mất xích (theo mô hình “GV giữ lõi, mỗi SV một mắt xích”) — áp dụng cho Hướng A:

Thành viên	Phạm vi sở hữu (để chấm & bảo vệ độc lập)
Thành viên khác	Lỗi runtime μEDP + tích hợp inference engine vào scheduler + thiết kế & đo benchmark (latency/jitter/footprint) so baseline.
Nguyễn Hoàng Hải Minh	Pipeline AI (thu thập dữ liệu → train → convert → eval) + ứng dụng demo + mở rộng công cụ μE-LS/tự động hóa.

7. Khung báo cáo chi tiết (mẫu cho khóa luận)

Yêu cầu em viết báo cáo theo khung dưới đây. Bốn câu hỏi bắt buộc phải trả lời rõ: (1) Bài toán & động lực — vì sao không dùng thẳng QP/C? (2) Đây là phần MỚI thật sự? (3) Kế hoạch đo đạc thực nghiệm? (4) Phân công 2 người?

Chương 1 — Mở đầu

- Bối cảnh & động lực: vì sao chọn mô hình event-driven/Active Object cho hệ nhúng; vì sao cần một runtime riêng thay vì dùng thẳng QP/C hoặc FreeRTOS.
- Mục tiêu, phạm vi, giới hạn của khóa luận.
- Tuyên bố đóng góp (contributions) — liệt kê rõ ràng, đặc biệt phần mới & phần AI.

Chương 2 — Cơ sở lý thuyết & khảo sát

- Active Object, Run-to-Completion, event-driven vs RTOS.
- Khảo sát: QP/C, FreeRTOS, Zephyr; framework TinyML (TFLite Micro, CMSIS-NN).
- Định vị μEDP trong bức tranh chung (bảng so sánh có trích dẫn).

Chương 3 — Phân tích & thiết kế μEDP

- Kiến trúc 3 tầng (App – Core – PAL).
- Các module lõi: scheduler ưu tiên, message pool, FSM/TSM, ISR bridge, logging pipeline.
- Toolchain μE-LS (Kconfig → codegen). Chỉ rõ điểm khác biệt/dóng góp so QP/C.

Chương 4 — Tích hợp AI

- Bài toán AI & bộ dữ liệu; kiến trúc mô hình; pipeline train → lượng tử hóa → deploy.
- Cách nhúng inference vào μEDP như một Active Object task; ràng buộc bộ nhớ/thời gian.
- (Nếu Hướng B/C: mô tả module AI tương ứng.)

Chương 5 — Hiện thực & Sản phẩm demo

- Phần cứng (board, cảm biến), sơ đồ hệ thống.
- Hiện thực phần mềm; hướng dẫn build & nạp.
- Kịch bản demo end-to-end (đầu vào → AI → đầu ra).

Chương 6 — Thực nghiệm & Đánh giá

- Phương pháp đo: latency, jitter, footprint RAM/Flash; accuracy/precision-recall của AI.
- So sánh baseline (FreeRTOS / bare-metal); bảng số liệu + biểu đồ.
- Phân tích, bàn luận, giới hạn.

Chương 7 — Kết luận & hướng phát triển

- Tóm tắt đóng góp đạt được so với mục tiêu.
- Hướng mở rộng (μE-OS, thêm nền tảng, mô hình AI lớn hơn...).

Phụ lục & Tài liệu tham khảo

- Link repo (có đủ port + lịch sử git thật), hướng dẫn tái lập.
- Bảng phân công công việc 2 thành viên.
- Tài liệu tham khảo theo chuẩn IEEE.

8. Tiêu chí nghiệm thu (checklist)

Đề tài được xem là **ĐẠT** khi hoàn thành toàn bộ nhóm tiêu chí bắt buộc bên dưới. Các mức **KHÁ / GIỎI – XUẤT SẮC** cộng thêm theo chiều sâu thực nghiệm và độ mới.

A. Nhóm tiêu chí BẮT BUỘC (điều kiện ĐẠT)

#	Tiêu chí	Minh chứng
A1	Sản phẩm demo chạy hoàn chỉnh trên phần cứng thật (end-to-end)	Video demo + hướng dẫn tái lập
A2	Có thành phần AI hoạt động thực tế (không chỉ mô tả)	Mô hình + kết quả inference on-device
A3	Mã nguồn build được, repo đầy đủ port + lịch sử git thật	Repo public + CI/hướng dẫn build
A4	Có thực nghiệm đo đặc định lượng	Bảng latency/footprint/accuracy
A5	Báo cáo đúng khung (Mục 7), trích dẫn đầy đủ	Bản báo cáo hoàn chỉnh
A6	Phân công 2 thành viên rõ ràng, bảo vệ độc lập được	Bảng phân công + vấn đáp

B. Nhóm tiêu chí NÂNG CAO (KHÁ → GIỎI/XUẤT SẮC)

#	Tiêu chí	Mức
B1	So sánh baseline nghiêm túc (FreeRTOS/bare-metal) với phân tích thống kê	Khá
B2	Chỉ ra & chứng minh được đóng góp MỚI so với QP/C	Giỏi
B3	Đa nền tảng: chứng minh porting thật ≥2 MCU khác kiến trúc	Giỏi
B4	Kết quả đủ tầm viết bài báo/hội nghị (novelty + benchmark)	Xuất sắc

Nguyên tắc chấm (nhắc nhóm)

Trọng số lớn nhất KHÔNG nằm ở “viết được bao nhiêu dòng” mà ở: sản phẩm chạy được, số liệu thực nghiệm trung thực, và trả lời được “điểm mới của em là gì”. Tránh tuyên bố “vượt/tốt hơn” khi chưa có số đo.

9. Kết luận & bước tiếp theo

μEDP là một nền tảng **đủ mạnh và đủ chín** để phát triển thành khóa luận tốt nghiệp. Việc cần làm là **bọc nó thành một sản phẩm biên có AI, chạy demo được, và chứng minh bằng số liệu**. Đề nghị em triển khai theo thứ tự:

- Chốt hướng** (khuyến nghị Hướng A) và bài toán AI + phần cứng cụ thể.
- Dọn repo**: push đủ các port MCU + khôi phục lịch sử git thật.
- Viết đề cương** theo khung Mục 7 (đặc biệt Chương 1 & 4) để GV duyệt trước khi code phần AI.
- Dựng pipeline thực nghiệm** sớm (đo baseline trước) để có số liệu xuyên suốt.
- Phân công** 2 thành viên theo Mục 6.4 và thống nhất mốc thời gian với GV.

Phiếu nhận xét lập ngày 14/08/2026 — dùng làm cơ sở trao đổi và định hướng, có thể điều chỉnh theo thống nhất giữa GV và nhóm sinh viên.